

Documentation Guide

Where to look, depending on what you need to do.

Status: Week 1 of 5 — foundation, data model, core services endpoints

Tests: 705 passing, 92% coverage

API: 42

Fastest way to see it running

1. Install [Docker Desktop](#), then from the repository root: `docker compose up -d --build`
2. Open `http://localhost:8000/docs` in a browser — this is the interactive API, generated automatically from the code, so it can never be out of date.
3. Load some sample data: `docker compose --profile seed up seed`

Log in with: `admin@transfleet.com` / `SmartLogistics!2026` — or use one of the other 10 seeded accounts (warehouse operator, courier, support) to see what each role can and cannot do.

If you want to...

GOAL	READ	WHAT'S THERE
Set the project up and run it	README.md	Step-by-step setup for macOS, Linux and Windows, or a single Docker command. Also covers running tests, seeding data and everyday commands.
Try the API myself	README.md → API reference	Points to the interactive Swagger UI, a Postman collection, and a browsable HTML page with copy-paste curl commands for every endpoint.
Understand how it's built and why	docs/ARCHITECTURE.md	System design, module boundaries, 16 documented design decisions. Section 1.1 maps every requirement from the assignment brief to what's actually been built, with an honest status for each.
See the database design	docs/database.md	Every table and column explained in plain language, with diagrams showing how they relate to one another.
Review what was actually delivered this week	docs/phase_1.md	The build log: what was built, defects found and fixed along the way, and what's intentionally left for later weeks.
Deploy this to a server or the cloud	deploy/README.md	Running in Docker, and deploying to Kubernetes on AWS (EKS) or Azure (AKS).
See the original requirements	docs/projectDocs/	The assignment briefs this project is built against, unmodified.

Generated for SmartLogistics — Intelligent Supply Chain & Delivery Orchestration Platform (TransFleet). All documents referenced above live in the project repository and stay in sync with the code — none of this is written by hand and left to go stale.